Truly Pre-Routing Timing Prediction via Considering Power Delivery Network

Yuyang Ye[1], Mingwei He[2], Lizheng Ren[2], Jianwang Zhai[3,†], Tinghuan Chen[4], Jun Yang[2], Longxing Shi[2]
[1]CUHK [2]Southeast University [3]Beijing University of Posts and Telecommunications [4]CUHK, Shenzhen

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133278

***Abstract*—Fast and accurate pre-routing timing prediction is essential in the chip design flow. However, existing machine learning (ML)-assisted pre-routing timing methods often overlook the impact of power delivery networks (PDNs), which contribute to IR drop and routing congestion. This limitation can make these methods less practical for real-world circuit design flows. To address this, we propose two specialized encoders—an IR drop-aware encoder and a routing congestion-aware encoder—that effectively capture PDN effects through multimodal fusion of netlist, layout, and PDN data. To mitigate the challenges of imbalanced multimodal fusion, we further develop a Pareto optimization approach to ensure balanced utilization of all modalities, enhancing timing prediction accuracy. Comprehensive experiments on large-scale open-source designs using TSMC's 16nm technology node validate the superiority of our model over state-of-the-art pre-routing timing prediction methods.**

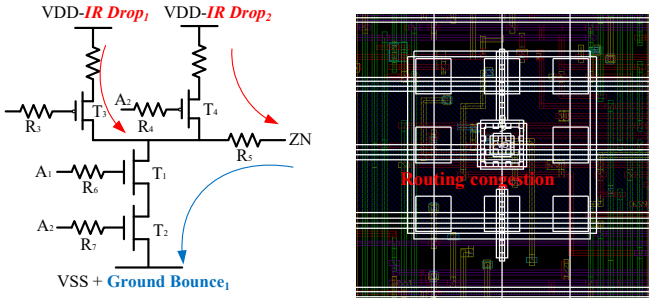
I. Introduction

Static Timing Analysis (STA) engines are widely used throughout physical design stages to guide timing optimization. However, during the placement stage, the absence of wire parasitic information leads to inaccurate timing estimates, resulting in suboptimal placement quality and requiring multiple design iterations. These iterations are often time-consuming due to the lengthy routing stages. Therefore, an accurate and efficient pre-routing timing prediction model based on the placement solution is crucial for enabling timing-driven placement and avoiding redundant routing [1].

Recent research has leveraged machine learning (ML) techniques to improve pre-routing timing prediction by incorporating the impact of routing on timing results [2]–[6]. Works like [5] and [6] focus on learning bimodal information from netlists and layouts using traditional multimodal fusion methods. Compared to unimodal approaches [2]–[4], incorporating multimodal information significantly improves the accuracy of pre-routing timing predictions.

However, these studies do not account for the impact of power delivery networks (PDNs) on timing performance. PDNs are essential for supplying stable voltage to chip components and maintaining signal integrity. As technology scales down, increased parasitic resistance in PDN interconnects can lead to IR drop issues, causing timing degradation. As shown in Fig. 1a, IR drop, including drop on the VDD rail and bounce on the VSS rail of a NAND cell, slows the switching speed and increases propagation delay. In traditional IC design flows, PDN design is done before placement. As shown in Fig. 1b, the white areas represent the PDN in the layout. A well-designed PDN requires dedicated metal layers that consume routing resources, increasing routing congestion. This reduces available space for signal routing, potentially leading to longer wire lengths and introducing additional parasitic resistance and capacitance, further impacting timing. Therefore, PDN effects, including IR drop and routing congestion, must be considered for achieving accurate pre-routing timing prediction models.

PDNs, netlists, and layouts represent distinct modalities in circuit design. However, the information in PDNs is more domain-specific and limited compared to netlists and layouts, creating an imbalance in modality representation. Traditional multimodal learning techniques, such as those in [5] and [6], assume a balanced relationship among modalities, which is not the case here. This imbalance presents challenges in training effective timing models, a problem we term imbalanced multimodal learning. A critical question arises: **How can we effectively integrate all circuit modalities (netlist, layout, and PDN) for pre-routing timing prediction?**



(a) Cell circuits. (b) Routed layout.

Fig. 1 PDN causes different IR drops and routing congestion results. IR drops influence cell delays and routing congestion results influence routing solutions.

In this work, we propose a novel pre-routing timing prediction framework that effectively balances the multimodal fusion of netlist, layout, and PDN data. To fully capture circuit information from all modalities, we develop two specialized encoders: IR drop-aware encoder integrates layout-based and PDN-based knowledge on one netlist-based model; Routing congestion-aware encoder integrates netlist-based and PDN-based knowledge on one layout-based model. To ensure balanced learning across all circuit modalities during timing model training, we introduce a Pareto multimodal fusion algorithm. This algorithm uses IR drop and routing congestion predictions as innocent assistance to improve the accuracy of pre-routing timing predictions. We highlight our contributions:

- We introduce the first multimodal fusion framework combining netlist, layout, and PDN information for pre-routing timing prediction.
- We develop an IR drop-aware encoder and a routing congestion-aware encoder, enabling accurate modeling of PDN impacts.
- We design the Pareto multimodal fusion algorithm to address the imbalanced multimodal learning problem during timing model training, thereby improving timing prediction accuracy with the assistance of IR drop and routing congestion predictions.
- We conduct experiments and ablation studies on large-scale open-source designs in the advanced TSMC 16nm technology.

II. Preliminaries

A. Power Network Design vs. PDN Graph

As shown in Fig. 2, we model a complex power delivery network as a PDN graph $\mathbb{G}_d = (\mathcal{V}_d, \mathcal{E}_d, \mathcal{P}_d)$, where each node $v_d \in \mathcal{V}_d$

†Corresponding author.

represents a PDN node, and each edge $e_d \in \mathcal{E}_d$ represents a power wire resistance or a power via resistance between different PDN nodes. The PDN path p_d in path set $\mathcal{P}_d$ is defined as:

Definition 1 (PDN path)**.** *The path from the power source to the target cell consists of all nodes, power vias, and power wires visited.*

In our work, PDN paths are selected based on the total resistance of power vias and power wires. For each cell, the top 5 paths with the smallest total resistance from each power source are identified using depth-first search.

B. Problem Formulation

Problem 1 (Pre-routing Timing Prediction Considering PDN)**.** *Given the pre-routing layout, netlist, and power delivery network of a circuit, our goal is to make a truly accurate and efficient estimation of the sign-off global timing metrics, i.e., endpoint arrival time.*

C. Multimodal Pre-routing Timing Prediction

As illustrated in Fig. 3a, we briefly introduce the overall flow of past mutimodal pre-routing timing prediction works [5], [6]. In [5], the authors introduce a novel endpoint embedding framework that combines graph and convolutional neural networks to extract netlist and layout information, respectively. Meanwhile, [6] presents LayNet, a multimodal neural network designed for pre-routing congestion prediction, which integrates both layout and netlist data through a novel heterogeneous message-passing approach.

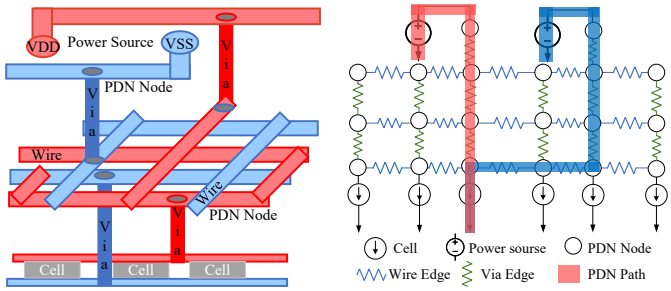
However, these approaches do not address the effects of PDNs, including IR drop and routing congestion, which are critical factors in accurate timing prediction. In contrast, this work aims to fill this gap by introducing a multimodal fusion framework that considers PDN effects alongside netlist and layout information. Our approach models timing performance by integrating data from three key aspects:

- **Netlist:** Netlist information is essential for delay computation, as it captures the connections between cells and wires, which form the basis for timing analysis.
- **Layout:** Placement and layout information play a crucial role in determining the routing solution, impacting timing performance.
- **PDN:** The PDN information contributes to model PDN effects, including IR drop and routing congestion results.

III. Proposed Framework

A. Overall Flow

As illustrated in Fig. 3b, we first briefly introduce the overall flow of our pre-routing timing prediction framework. The proposed framework can be divided into two parts: Part 1 learns the information in netlist, layout and PDN modality data through two specialized encoders: the IR drop-aware encoder (Section III-C) and the routing congestion-aware encoder (Section III-D); and Part 2 achieves balanced multimodal fusion based on Pareto method to improve timing prediction accuracy (Section III-E). In Part 1, the IR drop-aware encoder achieves a netlist-based model, while the routing-congestion encoder achieves a layout-based model. They learn netlist, layout, and PDN modality data. As shown in Fig. 3b, they can output embedding results, including $\boldsymbol{e}^{ir}$ and $\boldsymbol{e}^{co}$, which are used in IR drop and routing congestion prediction, respectively. By combining them ($\boldsymbol{e}^{ir}||\boldsymbol{e}^{co}$), it is used in pre-routing timing prediction. In Part 2, we develop a Pareto-based multimodal fusion method that ensures balanced integration of all modalities, improving the accuracy of the timing predictions. During model training, the method computes the optimal gradients for the IR drop-aware encoder (i.e., $\mathbf{p}^{ir}(\theta^{ir})$) and the routing congestion-aware encoder (i.e., $\mathbf{p}^{co}(\theta^{co})$). As shown in Fig. 3b, these gradients are used to update the encoder parameters θ^{ir} and θ^{co}, allowing the framework to integrate the IR drop and routing congestion predictions to enhance the pre-routing timing prediction. By fully leveraging and balancing the information from netlist, layout, and PDN modalities, our framework achieves highly accurate pre-routing timing predictions.



(a) Power delivery network (PDN). (b) PDN grpah.

Fig. 2 PDN is represented with PDN graph in our work. PDN node is translated to the node in graph, and PDN wire and via are translated to edge in graph. PDN path is the most important supplying path from the power source to the target cell.

As shown in Fig. 3a, traditional timing-driven multimodal learning methods often rely on a single multimodal joint timing loss, which optimizes the netlist and layout feature encoders. However, the PDN modality, being more domain-specific, can cause imbalanced learning when fused with netlist and layout data, leading to suboptimal results. To resolve this, we introduce IR drop loss and routing congestion loss in the respective encoders to balance modality contributions and improve prediction accuracy. In our work, the final loss functions are:

$$\mathcal{L} = \mathcal{L}_t + \mathcal{L}_{ir} + \mathcal{L}_{co}, \tag{1}$$

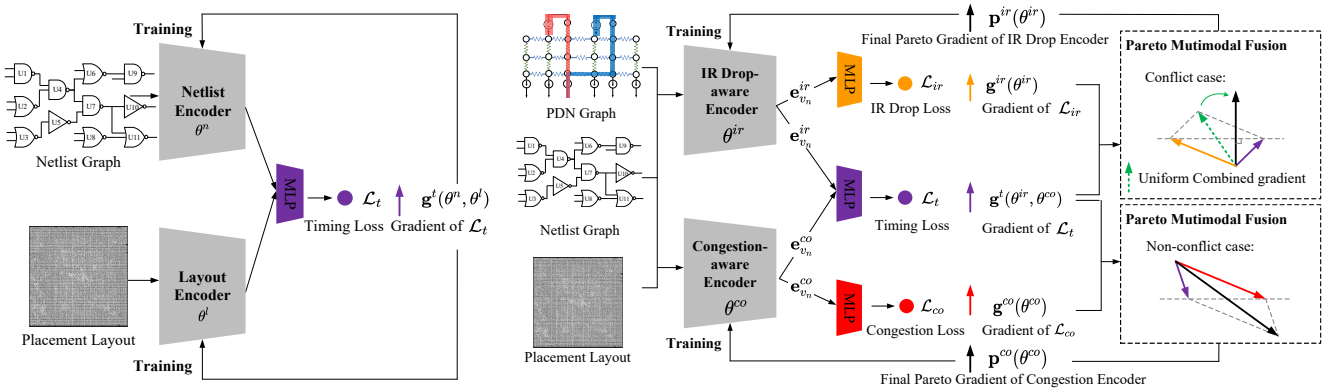
where $\mathcal{L}_t$ is the multimodal joint timing loss, $\mathcal{L}_{ir}$ is the IR drop loss and $\mathcal{L}_{co}$ is the routing congestion loss. They are cross-entropy loss functions. The details of achieving and updating IR drop-aware and routing congestion-aware encoders are discussed as follows.

B. Data Representation

PDN Modality Data. The PDN graph $\mathbb{G}_d$ is represented with node feature matrix $\boldsymbol{X}^D$: $\{\boldsymbol{x}^D_{v_d}, v_d \in \mathcal{V}_d\}$. The details of features in feature vector $\boldsymbol{x}^D_{v_d}$ includes: (1) Located metal layer: the number of the located metal layer. (2) Connected power wire resistance: total resistance of power wires connected with the target power node. (3) Connected power via resistance: total resistance of power vias connected with the target power node.

Netlist Modality Data. We transfer the circuit netlist to a graph $\mathbb{G}_n = (\mathcal{V}_n, \mathcal{E}_n)$ consisting of a node set ($\mathcal{V}_n$), an edge set ($\mathcal{E}_n$). Nodes are cells and Edges are wires in circuits. The details of features in node feature vector $\boldsymbol{x}^N_{v_n}$ for cell node include: (1) Cell type: e.g., NAND, NOR, embedded as a one-hot vector; (2) Pin capacitance: extracted from the timing library; (3) Cell drive strength: extracted from the cell type name of circuits. (4) Cell supply voltage: extracted from the design requirement. The details of features in edge feature vector $\boldsymbol{x}^N_{e_n}$ for wire include: (1) Placement wire delay: extracted from timing report of circuit netlists after placement. The wire delay is calculated based on Elmore delay and the Manhattan distance between the positions of a wire's drive pin and its sink pin.

Layout Modality Data. We divide the overall placement layout into 512×512 bins. For each bin b_l on layout, the detailed considered layout features $\boldsymbol{X}^L$: $\{\boldsymbol{x}^L_{b_l}, b_l \in 512 \times 512 \text{ bins}\}$ should be closely correlated with the final routing congestion and solution, which



(a) Past bi-modal pre-routing timing prediction work. (b) Our multi-modal pre-routing timing prediction work.

Fig. 3 Comparsion between our multimodal flow and conventional bimodal flow.

include: (1) The macro cell regions; (2) Rectangular uniform wire density (RUDY); (3) Gate density.

Labels. In our work, we use three distinct types of labels: (1) IR Drop Labels: The static IR drop values for each cell instance are extracted from ANSYS RedHawk [7], an industry-standard IR drop analysis tool. (2) Routing Congestion Labels: The golden congestion results of layouts are generated by IC Compiler II [8]. (3) Timing Labels: These labels represent the output pin arrival times for each cell after routing in the circuit. The golden IR drop-aware timing results are produced by the static timing analysis engine, PrimeTime [9]. The IR drop is incorporated into the timing analysis using the command read_sdf Modified_IR. sdf, where Modified_IR. sdf is the file generated by RedHawk after performing IR drop analysis.

C. IR Drop-aware Encoder

As shown in Fig. 4a, our IR drop-aware encoder learns timing-driven information for each cell v_n in the netlist through three distinct embedding strategies: (1) Netlist-only embedding: This embedding captures basic timing information from the circuit netlist by aggregating features of cells and wires. (2) Netlist-PDN embedding: This embedding gathers PDN information from the PDN paths associated with each cell in the netlist. (3) Netlist-layout embedding: This embedding integrates layout location data for each cell in the netlist through a novel message-passing paradigm applied to the layout edges. By combining these three embeddings, our model effectively incorporates a variety of timing-related information and output the IR drop-aware embedding result $\boldsymbol{e}^{ir}_{v_n}$, enhancing the accuracy of IR drop prediction and pre-routing timing prediction.

Netlist-only Embedding. Given the input netlist cell features $\{\boldsymbol{x}^N_{v_n}, v_n \in \mathcal{V}_n\}$ and edge features $\{\boldsymbol{x}^N_{e_n}, e_n \in \mathcal{E}_n\}$, graph learning can help achieve netlist feature embedding and output $\{\boldsymbol{e}^N_{v_n}, v_n \in \mathcal{V}_n\}$. For cell v_n, the $\boldsymbol{e}^N_{v_n}$ is generated via:

$$\boldsymbol{e}^N_{v_n} = \text{GNN}(\{\boldsymbol{x}^N_{v_n}, v_n \in \mathcal{V}_n). \tag{2}$$

Here, GNN proposed in the past timing prediction work [10] is used in our work to learn cell features and edge features jointly.

Netlist-PDN Embedding. For each cell v_n in the circuit netlist, we use a depth-first search (DFS) method to select PDN paths (shown in Section II-A) and generate the PDN path set $\mathcal{P}^d_{v_n}$. Using PDN node features $\{\boldsymbol{x}^D_{v_d}, v_d \in \mathcal{V}_d\}$, we leverage Transformers to embed PDN features path by path, learning information from the PDN paths. For a given PDN path p^d, the $\boldsymbol{t}_{p^d}$ is generated via:

$$\boldsymbol{t}_{p^d} = \text{Transformer}(\{\boldsymbol{x}^D_{v_d}, v_d \in \mathcal{N}_{p^d}\}), \tag{3}$$

where $\mathcal{N}_{p^d}$ is the node set on PDN path p^d. In real chip design, the cell's IR drop is caused jointly by different PDN paths. Thus, after encoding all paths in the path set $\{p^d \in \mathcal{P}^d_{v_n}\}$, we need to combine these path embeddings $\{\boldsymbol{t}_{p^d}, p^d \in \mathcal{P}^d_{v_n}\}$ together. Different PDN paths have different contributions to the IR drop of the target cell. Thus, we adopt a PDN path attention mechanism α_{p^d} to learn the different weights:

$$\alpha_{p^d} = \frac{\exp(\text{LeakyReLU}(\boldsymbol{W}_1 \cdot \boldsymbol{t}_{p^d}))}{\sum_{p^d_k \in \mathcal{P}^d_{v_n}} \exp(\text{LeakyReLU}(\boldsymbol{W}_1 \cdot \boldsymbol{t}_{p^d_k}))}, \tag{4}$$

where $\boldsymbol{W}_1$ is the trainable weight vector. Based on attention mechanisms, the final PDN path embedding result and netlist-PDN embedding result of cell v_n should be generated via:

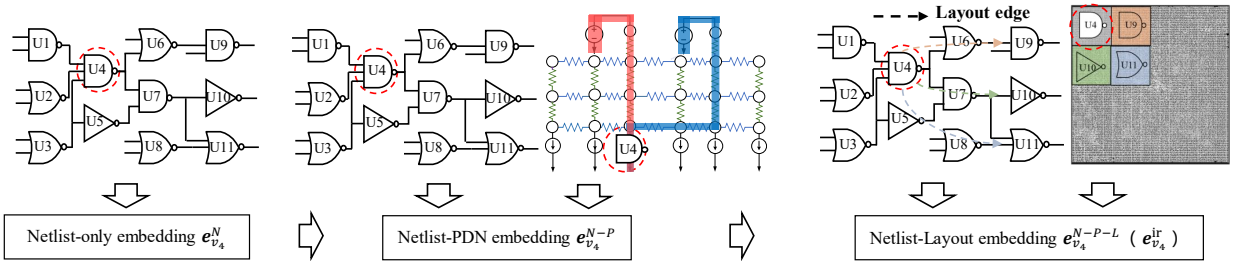
$$\boldsymbol{e}^P_{v_n} = \sigma(\sum_{p^d \in \mathcal{P}^d_{v_n}} \alpha_{p^d} \cdot \boldsymbol{t}_{p^d}), \quad \boldsymbol{e}^{N-P}_{v_n} = \boldsymbol{e}^P_{v_n} \| \boldsymbol{e}^N_{v_n}. \tag{5}$$

Netlist-Layout Embedding. Unlike traditional pre-routing timing prediction methods, our approach constructs layout edges for cells in the netlist based on their placement information. As shown in the right part of Fig. 4a, we provide an example of a layout edge. For cell U4, its layout neighbors U9, U10, and U11 are placed nearby, though not adjacent in the netlist. These layout neighborhoods are connected by layout edges, which are crucial for timing and IR drop prediction but challenging for conventional graph learning methods. We refer to these cells as layout neighborhoods of cell U4, connected by layout edges. To better capture this information, we introduce a layout neighbor attention mechanism ($\alpha^{N-L}_{v_n}$) that embeds cell information through layout edges. The layout attention coefficients $\alpha^{N-L}_{v_n}$ are computed as follows:

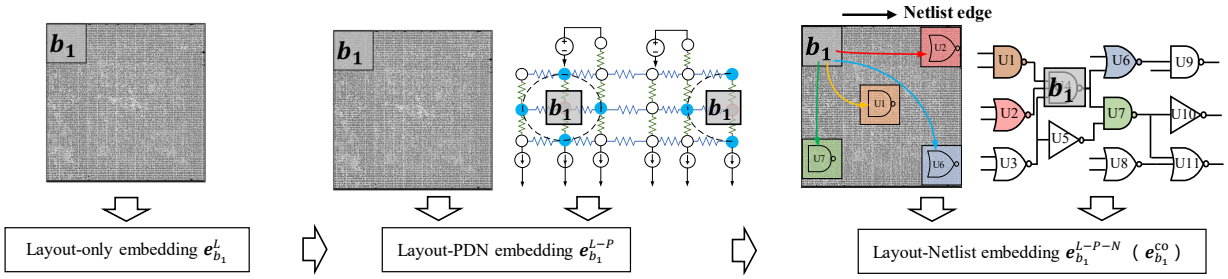
$$\alpha^{N-L}_{v_n} = \frac{\exp(\text{LeakyReLU}(\boldsymbol{W}_2 \cdot \boldsymbol{e}^{N-P}_{v_n}))}{\sum_{v_j \in \mathcal{N}^L_{v_n}} \exp(\text{LeakyReLU}(\boldsymbol{W}_2 \cdot \boldsymbol{e}^{N-P}_{v_j}))}, \tag{6}$$

where v_n is the target cell and the v_j is its layout neighbor belongs to layout neighborhood set N_{v_n}, which is connected with target cell v_n through layout edge. $\boldsymbol{W}_2$ is the trainable weight vector. The negative input slope of LeakyReLU nonlinear function is set as 0.2. Based on attention mechanisms, the final IR drop-aware embedding of cell v_n should be generated via:

$$\boldsymbol{e}^{ir}_{v_n} = \boldsymbol{e}^{N-P-L}_{v_n} = \sigma(\sum_{v \in \mathcal{N}_{v_n}} \alpha^{N-L}_{v} \cdot \boldsymbol{e}^{N-P}_{v}). \tag{7}$$



(a) The illustration of IR drop-aware encoder.



(b) The illustration of routing congestion-aware encoder.

Fig. 4 The information in the netlist graph, PDN graph, and layout image is learned through two encoders: IR drop-aware encoder and Routing congestion-aware encoder.

D. Routing Congestion-aware Encoder

As shown in Fig. 4b, in our routing congestion-aware encoder, for each bin b_l in the layout, we learn timing-driven information through three types of embeddings: (1) Layout-only embedding: Captures basic physical information from the circuit layout, important for modeling routing demands and solutions. (2) Layout-PDN embedding: Incorporates PDN information to predict routing congestion for each bin. (3) Layout-netlist embedding: Integrates netlist adjacent information for each bin in the layout.

Layout-only embedding. On layouts, We start by encoding layout features grid by grid independently. For the trade-off between efficiency and effectiveness, the ResNet [11] and ASPP [12] are used to extract and compress input layout features, respectively. Thus, the layout-only embedding result for each bin $\boldsymbol{e}^L_{b_l}$ can be computed as:

$$\boldsymbol{e}^L_{b_l} = \text{ResNet-ASPP}(\boldsymbol{x}^L_{b_l}). \tag{8}$$

Layout-PDN embedding. For each bin b_l, we generate a set of bin-wise PDN nodes based on their locations. As shown in the middle part of Fig. 4b, for bin b_l in the layout, two PDN nodes are located in the region, forming the bin-wise PDN node set. Using the PDN node features $\{\boldsymbol{x}^D_{v_d},\ v_d \in \mathcal{V}_d\}$, we apply a graph learning model (GraphSAGE [13]) to achieve local PDN feature embeddings and learn information from the PDN graph. For each PDN node v_d, the embedding $\boldsymbol{e}_{v_d}$ is generated as follows:

$$\boldsymbol{e}_{v_d} = \text{GraphSage}(\{\boldsymbol{x}^D_{v_d}, v_d \in \mathcal{N}^D_{b_l}\}), \tag{9}$$

where $\mathcal{N}^D_{b_l}$ is the PDN node set for the layout bin b_l. We then compute the layout-PDN embedding for bin b_l by averaging pooling the local feature embeddings of all PDN nodes in $\mathcal{N}^D_{b_l}$, and combining it with the layout-only embedding $\boldsymbol{e}^L_{b_l}$:

$$\boldsymbol{e}^{L-P}_{b_l} = \text{AVE}\ (e_{v_d}, v_d \in \mathcal{N}^D_{b_l}) \parallel \boldsymbol{e}^L_{b_l}. \tag{10}$$

Layout-Netlist embedding. Building on the netlist and layout, Lay-Net [6] integrates netlist-based message passing into a layout-based model to improve congestion prediction performance. As shown in the right part of Fig. 4b, netlist connection information can be modeled on the layout using netlist edges. For example, in bin b_1, where cell U4 is located, information from other bins containing neighboring cells of U4 can be captured through Lay-Net through netlist edge. Thus, the routing congestion-aware embedding $\boldsymbol{e}^{co}_{b_l}$ is computed as:

$$\boldsymbol{e}^{co}_{b_l} = \boldsymbol{e}^{L-P-N}_{b_l} = \text{Lay-Net}(\boldsymbol{e}^{L-P}_{b_l}). \tag{11}$$

Cell-wise Masking. Inspired by [5], gate-wise masking $\boldsymbol{M}_{v_n}$ can help us to get the routing congestion-aware embedding result for each gate $\{\boldsymbol{e}^{co}_{v_n},\ v_n \in \mathcal{V}_N\}$ as:

$$\boldsymbol{e}^{co}_{v_n} = \boldsymbol{M}_{v_n} \boldsymbol{e}^{co}_{b_l}. \tag{12}$$

IR drop prediction and routing congestion prediction are achieved based on the IR drop-aware embedding result $\boldsymbol{e}^{ir}_{v_n}$ and routing congestion-aware embedding result $\boldsymbol{e}^{co}_{v_n}$. Importantly, timing prediction is achieved through combining $\boldsymbol{e}^{ir}_{v_n}$ and $\boldsymbol{e}^{co}_{v_n}$.

E. Pareto Multimodal Fusion

As shown in Equation (1), timing loss, IR drop loss, and routing congestion loss are closely related in multimodal cases. However, their gradients may conflict during model training, as illustrated in Fig. 3b. This raises the challenge of effectively integrating these gradients. To address this, we introduce Pareto integration into our multimodal fusion framework, which resolves conflicts among the timing, IR drop, and routing congestion gradients. The Pareto optimization problem during gradient integration of training is formulated as follows:

$$\begin{aligned} \min_{\alpha^t, \alpha^{ir} \in \mathcal{R}} & \left\| \alpha^t \mathbf{g}^t(\theta^{ir}) + \alpha^{ir} \mathbf{g}^{ir}(\theta^{ir}) \right\|^2 \\ \text{s.t.} \quad & \alpha^t, \alpha^{ir} \geq 0, \alpha^t + \alpha^{ir} = 1, \end{aligned} \tag{13}$$

$$\begin{aligned} \min_{\beta^t, \beta^{co} \in \mathcal{R}} & \left\| \beta^t \mathbf{g}^t(\theta^{co}) + \beta^{co} \mathbf{g}^{co}(\theta^{co}) \right\|^2 \\ \text{s.t.} \quad & \beta^t, \beta^{co} \geq 0, \beta^t + \beta^{co} = 1, \end{aligned} \tag{14}$$

where $\|*\|$ denotes the L_2-norm. For the IR drop-aware encoder parameters θ^{ir}, $\mathbf{g}^t(\theta^{ir})$ and $\mathbf{g}^{ir}(\theta^{ir})$ denote the gradients of the multimodal joint timing loss $\mathcal{L}_t$ and the IR drop loss $\mathcal{L}_{ir}$, respectively. Similarly, for the routing congestion-aware encoder parameters θ^{co}, $\mathbf{g}^t(\theta^{co})$ and $\mathbf{g}^{co}(\theta^{co})$ represent the gradients of $\mathcal{L}_t$ and the routing congestion loss $\mathcal{L}_{co}$, respectively. The weights α^t, α^{ir}, β^t, and β^{co} are the final gradient weights to be determined.

This optimization problem involves finding the minimum norm in the convex hull of the gradient vectors ($\mathbf{g}^t$, $\mathbf{g}^{ir}$, and $\mathbf{g}^{co}$). The final Pareto solution provides a common descent direction that benefits all learning objectives. To illustrate solving the Pareto optimization problem in Equation (13), its analytical solution can be derived as:

$$\begin{cases} \alpha^t = 1, \alpha^{ir} = 0 & \cos\beta \geq \frac{\|\mathbf{g}^t(\theta^{ir})\|}{\|\mathbf{g}^{ir}(\theta^{ir})\|} \\ \alpha^t = \frac{\left(\mathbf{g}^{ir}(\theta^{ir}) - \mathbf{g}^t(\theta^{ir})\right)^\top \mathbf{g}^{ir}(\theta^{ir})}{\left\|\mathbf{g}^t(\theta^{ir}) - \mathbf{g}^{ir}(\theta^{ir})\right\|^2}, \alpha^{ir} = 1 - \alpha^t & \text{otherwise,} \end{cases} \tag{15}$$

where β is the angle between $\mathbf{g}^t(\theta^{ir})$ and $\mathbf{g}^{ir}(\theta^{ir})$. Then, we have the final gradient after integration:

$$\mathbf{p}^{ir}(\theta^{ir}) = 2\alpha^t \mathbf{g}^t(\theta^{ir}) + 2\alpha^{ir}\mathbf{g}^{ir}(\theta^{ir}). \tag{16}$$

Here we use 2α as the gradient weight to keep the same gradient weight summation with a uniform baseline (i.e., $2\alpha_t^{ir} + 2\alpha^{ir} = 2$). When using stochastic gradient descent based on the integrated gradient, the parameter of the IR drop-aware encoder is updated as:

$$\theta^{ir} := \theta^{ir} - \varepsilon \mathbf{p}^{ir}(\theta^{ir}) + \varepsilon\zeta, \tag{17}$$

where ζ is the stochastic gradient descent noise term and it satisfies:

$$\zeta \sim \mathcal{N}\left(0, \frac{\left(2\alpha^t\right)^2 C^t + \left(2\alpha^{ir}\right)^2 C^{ir}}{|S|}\right), \tag{18}$$

where $\frac{C^t}{|S|}$ and $\frac{C^{ir}}{|S|}$ represent batch sampling covariance. Studies [14], [15] show that the strength of noise in stochastic gradient descent influences the minima found, with larger noise often leading to flatter minima that generalize better. For both the IR drop-aware and routing congestion-aware encoders, the gradients of the IR drop loss and routing congestion loss typically have larger magnitudes and higher batch sampling covariance than the timing loss (i.e., $\mathbf{g}^{ir}(\theta^{ir}) \gg \mathbf{g}^t(\theta^{ir})$ and $\mathbf{g}^{co}(\theta^{co}) \gg \mathbf{g}^t(\theta^{co})$). Analytical Pareto methods assign greater weight to the timing loss gradient, which can weaken model generalization in multimodal learning. To address this, our method considers both the direction and magnitude during gradient integration. It ensures the final gradient aligns with all learning objectives while enhancing the strength of noise to improve generalization. We handle conflict and non-conflict cases separately:

Non-conflict case. We first consider the case where $\cos\beta > 0$. In this scenario, the cosine similarity between $\mathbf{g}^t(\theta^{ir})$ and $\mathbf{g}^{ir}(\theta^{ir})$ is positive. Since the gradient vectors' convex combination is common to all learning objectives, we assign $2\alpha_t^{ir} = 2\alpha^{ir} = 1$ instead of using the analytic Pareto solution for gradient integration. This choice helps to enhance the stochastic gradient descent noise term. With this setting, the final gradient can be formulated as:

$$\mathbf{p}^{ir}(\theta^{ir}) = \mathbf{g}^t(\theta^{ir}) + \mathbf{g}^{ir}(\theta^{ir}). \tag{19}$$

Conflict case. In the case where $\cos\beta < 0$, it is crucial to determine a common direction for all losses while enhancing the strength of the stochastic gradient descent noise term during gradient integration. First, we compute the analytic solution of Equation (15), which provides a non-conflicting direction. To further enhance the noise term strength, we increase the magnitude of the final gradient as:

$$\mathbf{p}^{ir}(\theta^{ir}) = \frac{2\alpha^t \mathbf{g}^t(\theta^{ir}) + 2\alpha^{ir}\mathbf{g}^{ir}(\theta^{ir})}{\|2\alpha^t \mathbf{g}^t(\theta^{ir}) + 2\alpha^{ir}\mathbf{g}^{ir}(\theta^{ir})\|} \cdot \left\|\mathbf{g}^t(\theta^{ir}) + \mathbf{g}^{ir}(\theta^{ir})\right\|. \tag{20}$$

In this case, the noise term satisfies:

$$\xi \sim \mathcal{N}\left(0, \lambda^2 \cdot \frac{\left(2\alpha^t\right)^2 C^t + \left(2\alpha^{ir}\right)^2 C^{ir}}{|S|}\right), \tag{21}$$

where $\lambda = \left\|\mathbf{g}^t(\theta^{ir}) + \mathbf{g}^{ir}(\theta^{ir})\right\| / \left\|2\alpha^t \mathbf{g}^t(\theta^{ir}) + 2\alpha^{ir}\mathbf{g}^{ir}(\theta^{ir})\right\|$. Thus, our noise term ξ has a larger strength than the noise term ζ of the analytical Pareto solution shown in Equation (18). Accordingly, model generalization is improved.

TABLE I Benchmark statistics.

Circuit	#gates	#wires	#EPs	#PNs	#PVs+#PWs	Δt
DMX	11616	11671	4671	6394	11706	12.2%
GFX	11004	11156	49403	99900	251314	13.4%
AC97	4954	5046	2262	3418	5435	8.5%
VGA	32727	32863	17749	37844	48510	16.5%
NOVA	105756	107641	30409	43912	94740	15.6%
Tot.Train	166057	168377	104494	191468	471705	66.2%
TOP	3943	4121	1694	2698	5163	9.2%
ECG	39992	40941	7691	11804	29095	14.2%
ETH	25327	25450	11681	9578	26314	11.9%
USB	6666	7000	1799	2298	5700	14.4%
TATE	153192	154720	31480	61666	114319	18.2%
Tot.Test	229120	232232	54345	88044	180591	67.9%

IV. Experimental Results

Our framework is implemented in Python using the PyTorch library, and in C++. The multimodal timing model is trained on a Linux machine with 32 cores and 4 NVIDIA Tesla V100 GPUs, completing training in approximately 4.5 hours using parallel training on 4 GPUs. The process uses 128GB of memory, with embedding dimensions for both IR drop-aware and routing congestion-aware encoders set to 128. The model is trained for 200 epochs with a learning rate of 0.001 and a batch size of 1024.

We train and evaluate our timing model using various open-source designs [16], and the framework can be applied to unseen designs without retraining. The benchmark circuits include designs with varying netlist and PDN scales. These open-source designs are synthesized using Synopsys Design Compiler [17] under the TSMC 16nm technology node and go through power planning, placement, and routing using Synopsys IC Compiler II [8]. The VDD options include $1.0V$, $0.9V$ and $0.8V$. Detailed information is shown in TABLE I, where #EPs represents the number of endpoints, #PNs represents the number of power nodes, #PVs+#PWs represents the total number of power vias and wires. Δt represents the percentage of timing degradation induced by IR drop.

A. Timing Prediction Accuracy

We compare our framework with previous state-of-the-art (SOTA) works [2]–[6] for pre-routing timing prediction on cell output pin arrive time. Unlike these methods, which struggle with PDN effects such as IR drop and routing congestion, our framework effectively addresses these challenges. In "Direct Fusion", multimodality data is learned directly without Pareto fusion and the timing model is trained based on timing loss. In "Analytical Pareto", multimodality data is learned with Pareto fusion using the analytical solutions to integrated gradients of timing, IR drop and routing congestion loss. The accuracy of our framework and other SOTA models is shown in TABLE II, evaluated using the R^2 score (higher is better) and maximum absolute error ($MAXE$, lower is better).

Our results demonstrate that our model accurately predicts true post-routing timing results while accounting for PDN effects. For training designs, the average R^2 and $MAXE$ scores are 0.96/5.30ps.

TABLE II Timing prediction accuracy. The unit of "$MAXE$" is ps.

Cir.	DAC19 [2]		DAC22-He [3]		DAC22-guo [4]		DAC23 [5]		Lay-Net [6]		Direct Fusion		Analytical Pareto		Ours	
	R^2	$MAXE$	R^2	$MAXE$	R^2	$MAXE$	R^2	$MAXE$	R^2	$MAXE$	R^2	$MAXE$	R^2	$MAXE$	R^2	$MAXE$
DMX	0.67	26.33	0.71	22.89	0.80	28.36	0.86	22.45	0.87	26.21	0.85	27.33	0.88	12.38	**0.96**	**5.26**
GFX	0.71	31.20	0.76	29.34	0.79	25.24	0.82	21.38	0.84	17.39	0.86	13.54	0.90	11.28	**0.93**	**4.52**
AC97	0.75	49.29	0.81	32.68	0.83	30.99	0.87	28.34	0.88	21.22	0.86	25.66	0.92	13.51	**0.94**	**3.06**
VGA	0.77	75.13	0.83	42.97	0.84	39.68	0.85	34.27	0.84	27.54	0.84	31.06	0.91	11.99	**0.97**	**6.59**
NOVA	0.71	93.14	0.81	71.54	0.82	38.17	0.85	56.28	0.87	37.28	0.88	32.59	0.93	18.46	**0.98**	**7.09**
Ave.	0.72	55.02	0.78	39.88	0.82	39.38	0.85	32.54	0.86	25.93	0.86	26.04	0.91	13.52	**0.96**	**5.30**
TOP	0.54	36.19	0.63	24.93	0.69	23.97	0.83	18.59	0.82	19.27	0.84	17.86	0.86	15.87	**0.96**	**3.08**
ECG	0.49	53.25	0.57	46.27	0.61	38.65	0.77	31.24	0.80	33.65	0.82	28.39	0.87	16.52	**0.93**	**4.53**
ETH	0.51	63.72	0.59	52.19	0.64	45.27	0.79	27.29	0.78	31.20	0.72	33.96	0.84	9.86	**0.91**	**3.67**
USB	0.38	46.86	0.56	50.28	0.63	41.38	0.76	32.05	0.80	24.36	0.73	33.81	0.86	12.56	**0.92**	**3.09**
TATE	0.33	113.64	0.62	99.37	0.65	95.68	0.80	78.69	0.81	69.59	0.75	81.22	0.83	22.53	**0.95**	**9.32**
Ave.	0.45	62.73	0.59	54.61	0.64	48.99	0.79	37.57	0.80	35.61	0.77	39.05	0.85	15.45	**0.93**	**4.74**

TABLE III Runtime Comparison. The unit is "s".

Cir.	Commerial Flow (20 threads)				Ours	
	Route [8]	IR [7]	STA [9]	Total	Total	Speedup
DMX	20329	77	18	20424	12.56	1626×
GFX	7782	67	16	7865	11.22	701×
AC97	618	72	8	698	4.27	164×
VGA	32727	88	16	32831	9.94	3303×
NOVA	75381	103	45	75529	25.89	2917×
TOP	298	43	9	350	6.17	57×
ECG	35689	72	17	35778	13.65	2621×
ETH	24659	97	12	24768	7.54	3285×
USB	518	67	9	594	5.12	116×
TATE	113192	218	68	113478	31.06	3653×
Ave.	31119	90	22	31231	12.74	1844×

For unseen testing designs, these scores are 0.93/4.74ps. After analyzing the results, we summarize our findings below:

- **Highly accurate:** Our framework achieves accurate pre-routing timing prediction for both seen and unseen circuits, generalizing well across designs of varying functions and scales without retraining. This means our model is practical in IC design flow.
- **Triple Modality Advantage:** By fully and effectively integrating netlist, layout, and PDN modality data, our framework outperforms prior single-modal models [2]–[4] and bi-modal approaches [5], [6].
- **Improved Generalization:** Compared with direct fusion and analytical Pareto fusion, our modified Pareto fusion significantly enhances generalization on unseen designs.

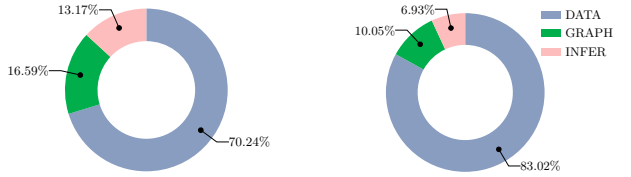
B. Runtime Comparison

We demonstrate the speedup of our model compared to an industry-leading commercial tool. The IR drop analysis and routing procedures in modern VLSI design flows are extremely time-consuming due to their high complexity. In contrast, machine learning models offer overwhelming efficiency advantages. The runtime results of our work, shown in TABLE III, include the model inference, graph construction and data preprocessing stage. On average, our proposed flow runs 1844× faster than the commercial tool for evaluating sign-off timing metrics. As illustrated in Fig. 5, the majority of the runtime in our framework is attributed to the data preprocessing stage. It means that there is still a large alteration potential.

C. Ablation Study

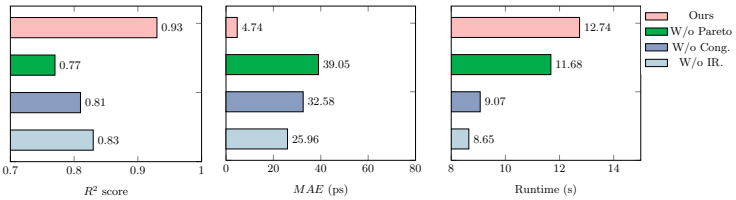
This section presents ablation studies. We compare the following schemes: (1) **W/o Pareto**: Multi-modality data fusion is performed without our proposed Pareto training method (Section III-E), relying solely on a single timing loss during training. (2) **W/o Cong.**: The routing congestion-aware encoder (Section III-D) is excluded from the framework. (3) **W/o IR**: The IR drop-aware encoder (Section III-C) is excluded from the framework.

As demonstrated in Fig. 6, our work is compared with other schemes by R^2 score (Fig. 6a), MAE (Fig. 6b) and runtime (Fig. 6c). The results highlight that the inclusion of multimodal data from



(a) On USB (small circuit). (b) On TATE (large circuit).

Fig. 5 Runtime breakdown of our work.



(a) (b) (c)

Fig. 6 Comparison among different schemes by (a) R^2 score, (b) $MAXE$, and (c) runtime.

netlist, layout, and PDN, processed through the IR drop-aware encoder and the routing congestion-aware encoder, is critical for accurate pre-routing timing prediction. Moreover, removing the Pareto multimodal fusion approach significantly degrades model accuracy, as the reliance on a single timing loss is insufficient to capture complex dependencies. In conclusion, the ablation study demonstrates the importance and necessity of the IR drop-aware encoder, routing congestion-aware encoder, and Pareto multimodal fusion method.

V. Conclusion

This work introduces a comprehensive pre-routing timing prediction framework that effectively integrates netlist, layout, and PDN modality data. The proposed IR drop-aware and routing congestion-aware encoders enable detailed modeling of timing information and PDN impacts. The Pareto multimodal fusion algorithm addresses imbalances among circuit modalities during the training of our multimodal timing model, ensuring meaningful contributions from each modality data. Experiments on advanced TSMC 16nm designs validate the framework, demonstrating significant improvements in timing prediction accuracy and efficiency.

Acknowledgment

This work is supported by the National Key R&D Program of China (2022YFB2901100), the National Natural Science Foundation of China (No. 62404021), and the Beijing Natural Science Foundation (No. 4244107, QY24216, QY24204).

REFERENCES

[1] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI physical design: from graph partitioning to timing closure*. Springer, 2011, vol. 312.

[2] E. C. Barboza, N. Shukla, Y. Chen, and J. Hu, "Machine learning-based pre-routing timing prediction with reduced pessimism," in *Proc. DAC*, 2019, pp. 1–6.

[3] X. He, Z. Fu, Y. Wang, C. Liu, and Y. Guo, "Accurate timing prediction at placement stage with look-ahead rc network," in *Proc. DAC*, 2022, pp. 1213–1218.

[4] Z. Guo, M. Liu, J. Gu, S. Zhang, D. Z. Pan, and Y. Lin, "A timing engine inspired graph neural network model for pre-routing slack prediction," in *Proc. DAC*, 2022, pp. 1207–1212.

[5] Z. Wang, S. Liu, Y. Pu, S. Chen, T.-Y. Ho, and B. Yu, "Restructure-Tolerant timing prediction via multimodal fusion," in *ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.

[6] S. Zheng, L. Zou, P. Xu, S. Liu, B. Yu, and M. Wong, "Lay-net: Grafting netlist knowledge on layout-based congestion prediction," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.

[7] Ansys, "RedHawk User Guide," https://www.ansys.com/products/semiconductors/ansys-redhawk-sc, 2024.

[8] Synopsys, "IC Compiler II User Guide," https://www.synopsys.com/implementation-and-signoff/physical-implementation/ic-compiler.html, 2023.

[9] ——, "PrimeTime User Guide," https://www.synopsys.com/cgi-bin/imp/pdfdla/pdfr1.cgi?file=primetime-wp.pdf, 2023.

[10] Y. Ye, T. Chen, Y. Gao, H. Yan, B. Yu, and L. Shi, "Graph-learning-driven path-based timing analysis results predictor from graph-based timing analysis," in *Proceedings of the 28th Asia and South Pacific Design Automation Conference*, 2023, pp. 547–552.

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[12] R. Azad, M. Asadi-Aghbolaghi, M. Fathy, and S. Escalera, "Attention deeplabv3+: Multi-level context attention mechanism for skin lesion segmentation," in *European Conference on Computer Vision (ECCV)*. Springer, 2020, pp. 251–266.

[13] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in neural information processing systems*, vol. 30, 2017.

[14] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," *arXiv preprint arXiv:1609.04836*, 2016.

[15] B. Neyshabur, S. Bhojanapalli, D. McAllester, and N. Srebro, "Exploring generalization in deep learning," *Advances in neural information processing systems*, vol. 30, 2017.

[16] OpenCores, http:///opencores.org/, 2023.

[17] Synopsys, "Design Compiler User Guide," https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html, 2024.